

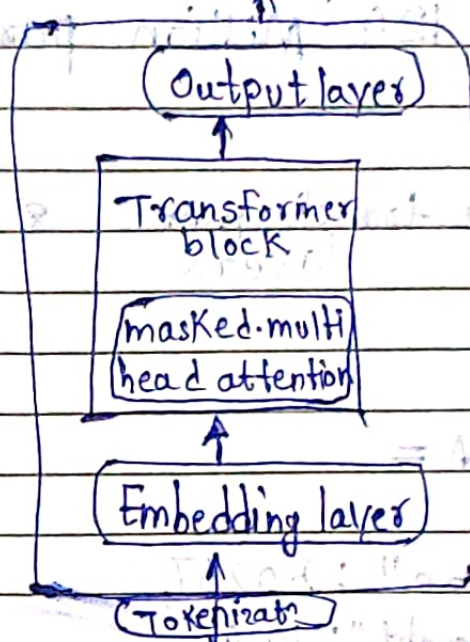
LLM Architecture

What we covered

'Every effort moves you forward!'

We implemented Attention module in prev. chap.

Embedding layer & Tokenization covered in chap 2

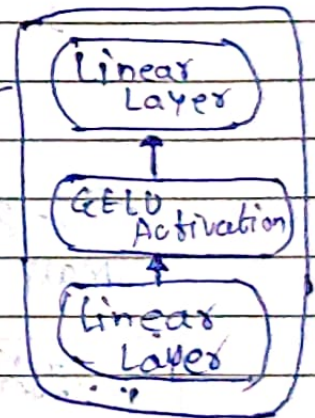
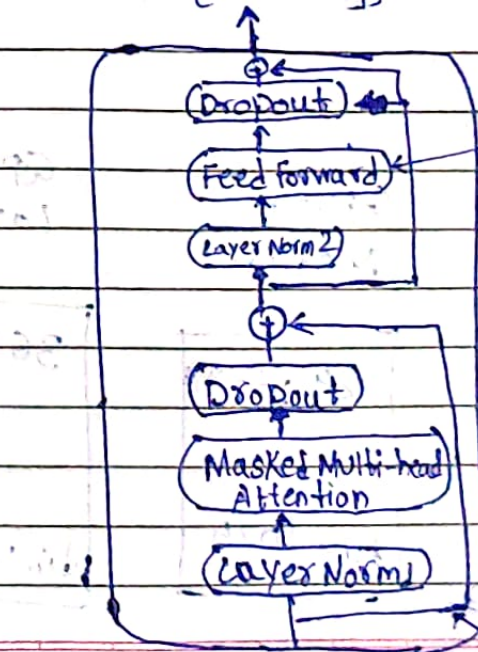


Transformer block are a Key component of GPT-like ~~the~~ LLM, which we are going to implement with some extra imp. concept

'Every effort moves you'

Zoom in Transformer block

output $\left[\begin{bmatrix} \vdots \\ \vdots \end{bmatrix} \right]$



A view into Feed Forward block

Input tokens $\left[\begin{bmatrix} \vdots \\ \vdots \end{bmatrix} \right]$

tokens X dimension

shortcut connect

We are going to use same Architecture used
For GPT-2

↳ 124 Million parameters

GPT-2 used

↳ 12 transformer layers & 124 Million Parameters

GPT-Config-124M =

```
{
  "vocab_size": 50257,
  "Context-length": 1024,
  "emb-dim": 768,
  "n-heads": 12,
  "n-layers": 12, # No. of layers
  "drop rate": 0.1,
  "qkv-bias": false, # Query-Key-Value bias
}
```

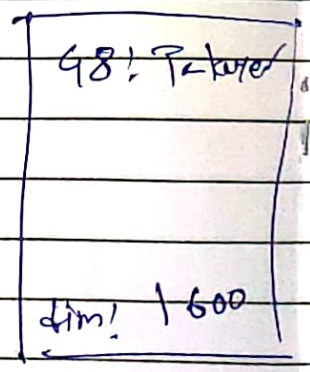
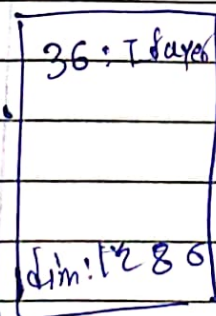
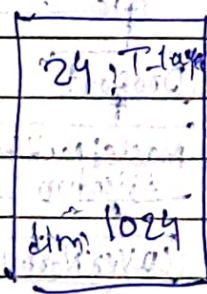
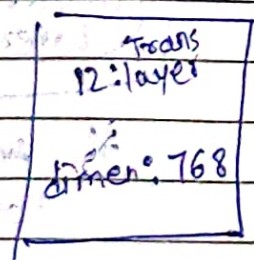
Random values in embedding are updated later through backpropagation

GPT-2 Small

GPT2 Medium

GPT3 Large

GPT4 Extra-large



Let's create GPT Placeholder Arch. / Dummy model

Revise

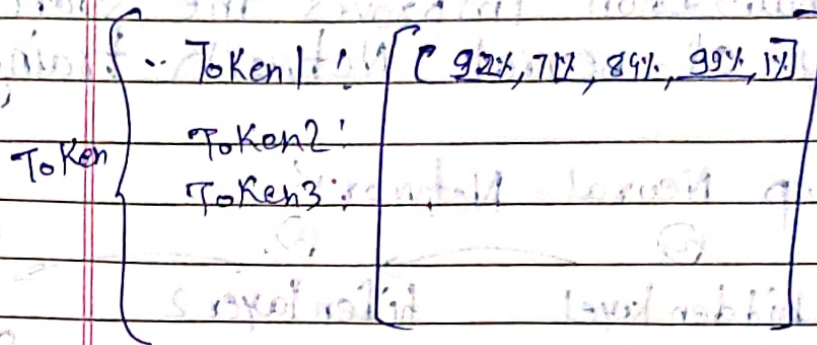
Token embedding

Position Embedding

There is 'logits' in code,

see

Output logits for 1 text sample



Vocab size = 50,257

Logits will tell which output token will have highest chance to be next one

Dummy model in VS code.

Final GPT architecture

Transformer block

2) Layer Normalization

3) GELU activation

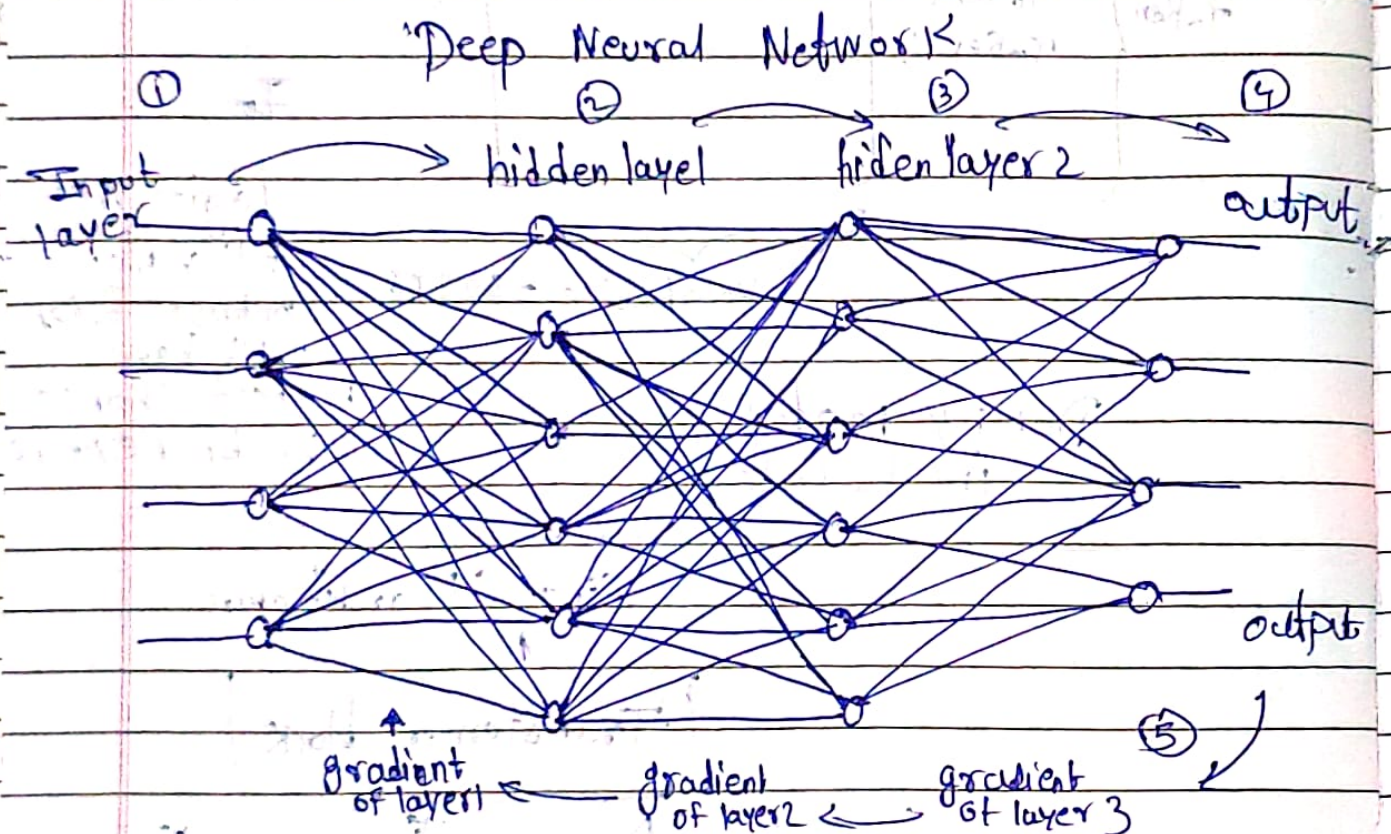
4) Feed forward Network

5) Shortcut connection

1) GPT backbone

* Layer Normalization (LayerNorm)

- 1] Training deep neural network with many layers can be challenging due to,
 - ↳ Vanishing / exploding gradients.
- 2] This leads to unstable training dynamics.
- 3] Layer Normalization improves the stability & efficiency of neural network training.



For every hidden layer, we have gradient for it

loss w.r.t parameters

based on output & gradient, we do back ward pass, & try to update these parameters

gradient of layer 1 depends on, output of first hidden layer 1

∴ Gradient Depends on large output

If the, Gradient is to in any intermediate ^{last} stage

Problem 1:

Small

large

↳ The time we reach first layer, input gradient will become so small.

↳ The time we reach first layer, gradient ~~is~~ could explode to very large value.

∴ Gradient Explosion
∴ Unstable training Dynamics

↳ We will not update parameters

↳ learning is going to stagnate
∴ Unstable training Dynamics

But, what if gradient dependent on output of layer,

so what if we control it?

Solution 1

↳ ^{so layer} ~~Batch~~ Normalization

, is used to keep gradient stable.

Problem 2

As training proceeds, input of each layer can change (internal covariate shift)

↓
This delays convergence

→ Layer Normalization ^{Prevent this}

9) Main Idea:

Adjust outputs of neural network to have mean zero & variance of one.

↳ This speeds up the convergence.

Lets ~~the~~ Suppose,

$[1.1, 0.8, 2.3, 4.9]$ ← These are outputs of x_1, x_2, x_3, x_4 in a many one hidden layer

$$\therefore \text{Mean} = \mu = \frac{x_1 + x_2 + x_3 + x_4}{4} = 2.15$$

$$\text{Variance} = \text{Var} = \frac{1}{4} [(x_1 - \mu)^2 + (x_2 - \mu)^2 + (x_3 - \mu)^2 + (x_4 - \mu)^2]$$

$$\text{Var} = 2.0025$$

$$\text{Normalized: } \left[\frac{x_1 - \mu}{\sqrt{\text{Var}}}, \frac{x_2 - \mu}{\sqrt{\text{Var}}}, \frac{x_3 - \mu}{\sqrt{\text{Var}}}, \frac{x_4 - \mu}{\sqrt{\text{Var}}} \right]$$

$$= [-0.74, -0.85, 0.1, 1.59]$$

$x_1 \quad x_2 \quad x_3 \quad x_4$

This is
New
output
of
hidden
layer

$$\text{Mean} = 0$$

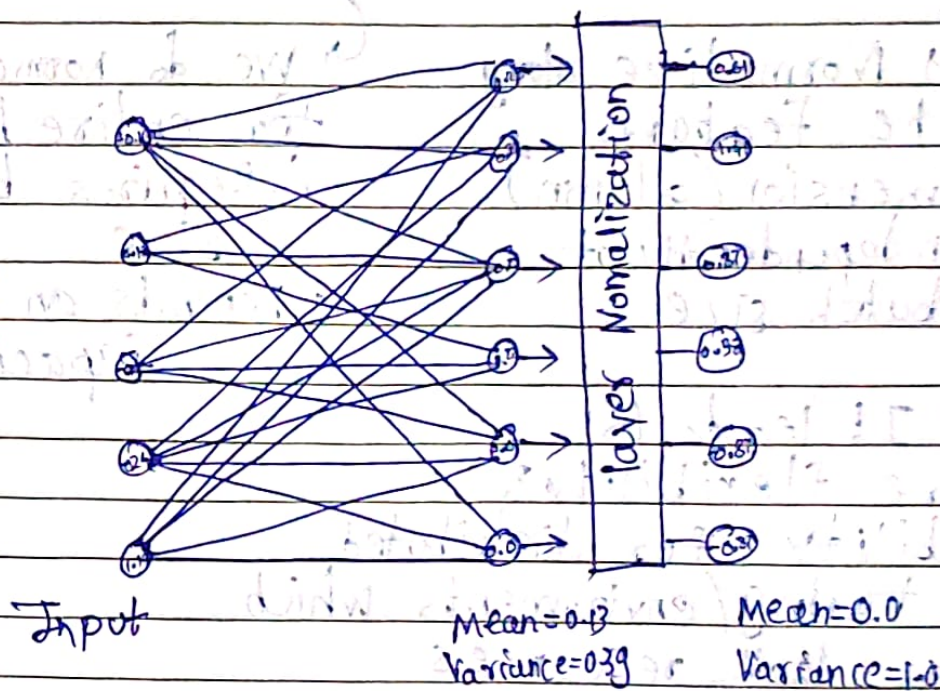
$$\text{Variance} = 1$$

↳ This
is whole
reason
for normal
zero

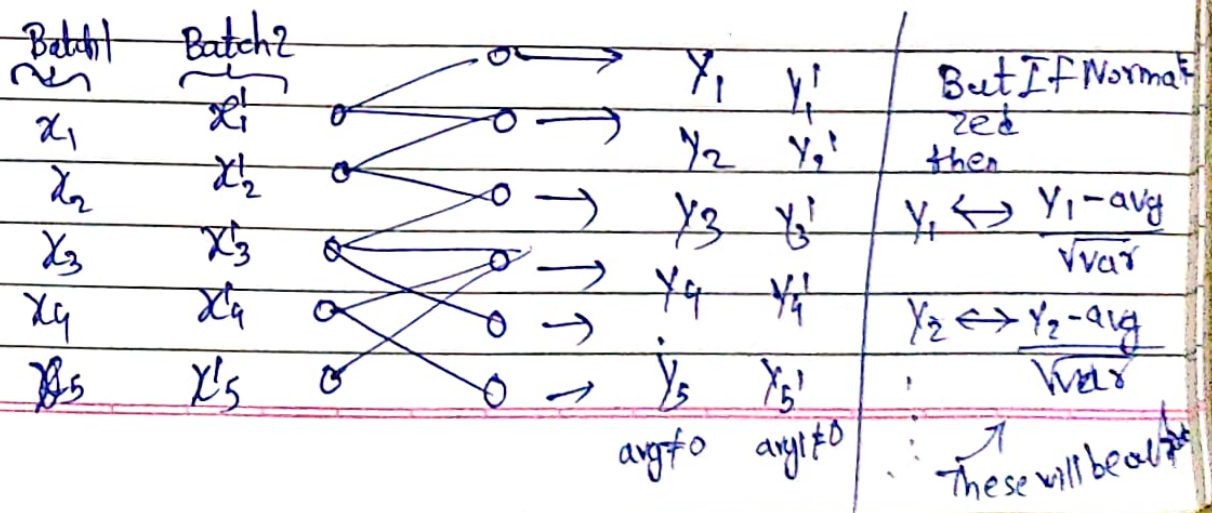
This solves the Internal covariate shift problem & provide stability.

* In GPT-2 & modern transformer architectures, layer Normalization is typically applied before & after the Multihead attention module & before the final output layer.

(The class of Normalization is outside, in program)



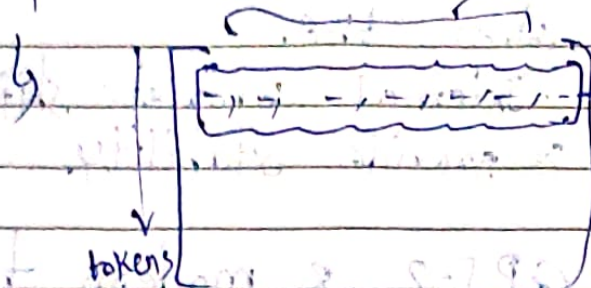
So suppose



* Layer Norm 1

So, our input is

Embedding size (768)



→ We will Normalize all the elements of row,

So that $\text{Avg} = 0$ $\text{var} = 1$

Code in C++

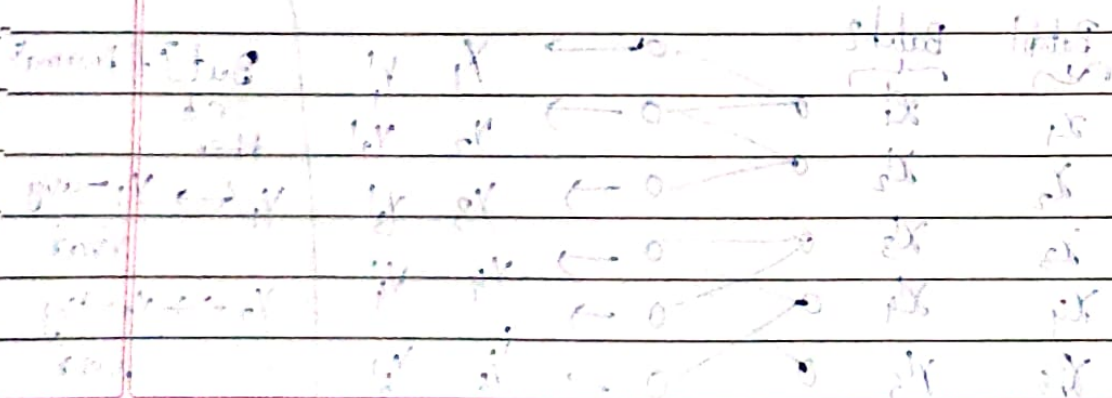
1) Layer Vs Batch Normalization

↳ Normalize along the Feature dimension (column) independently of batch size

↳ We do normalization for entire batch
∴ Requires batch size

∴ Depends on hardware Capacity

↳ It leads to more flexibility & Stability for distributed training/environments which lack resources



* GELU Activation

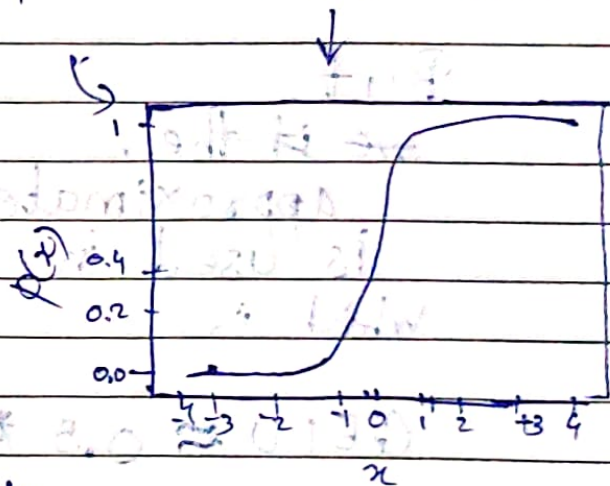
↳ We will implement a small neural Network Sub-model that is a parts of the LLM transformer block.

Two Activation Function commonly used implemented in LLM

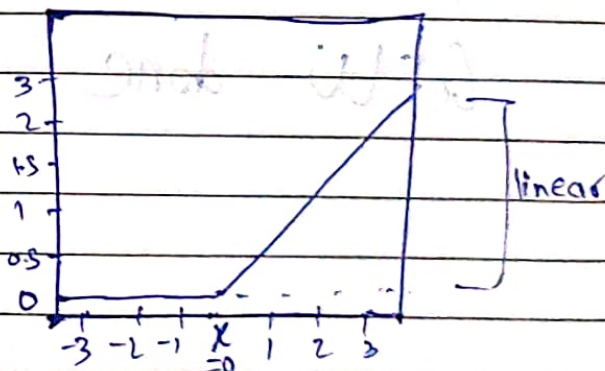
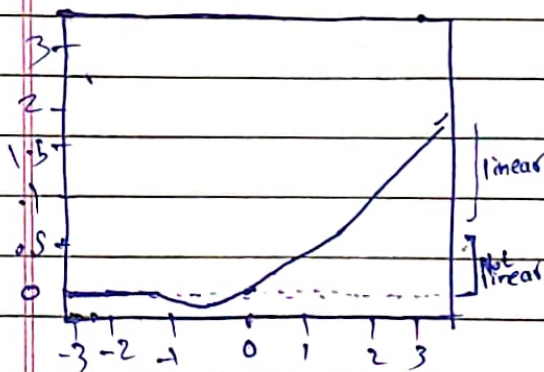
↳ GELU
↳ SwiGLU

$$\text{GELU} \rightarrow x * \phi(x)$$

→ CDF of standard Gaussian distribution



GELU vs RELU



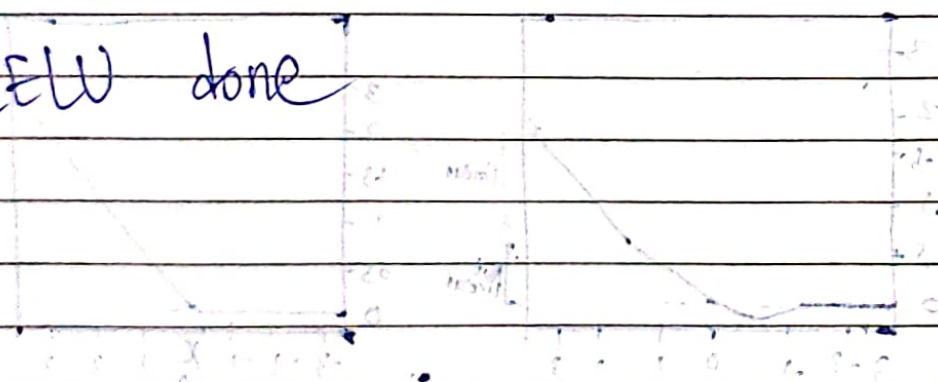
What are the adv. of GELU over ReLU

- 1) Continuous all over, where in ReLU $x=0$ is it is discontinuous, therefore Not differentiable
- 2) It is Not zero for negative x , that solves, the dead neuron problem, Even if output of neuron is negative, even if it goes through GELU, it will not become zero, it will still contribute to learning process, which not happen in ReLU

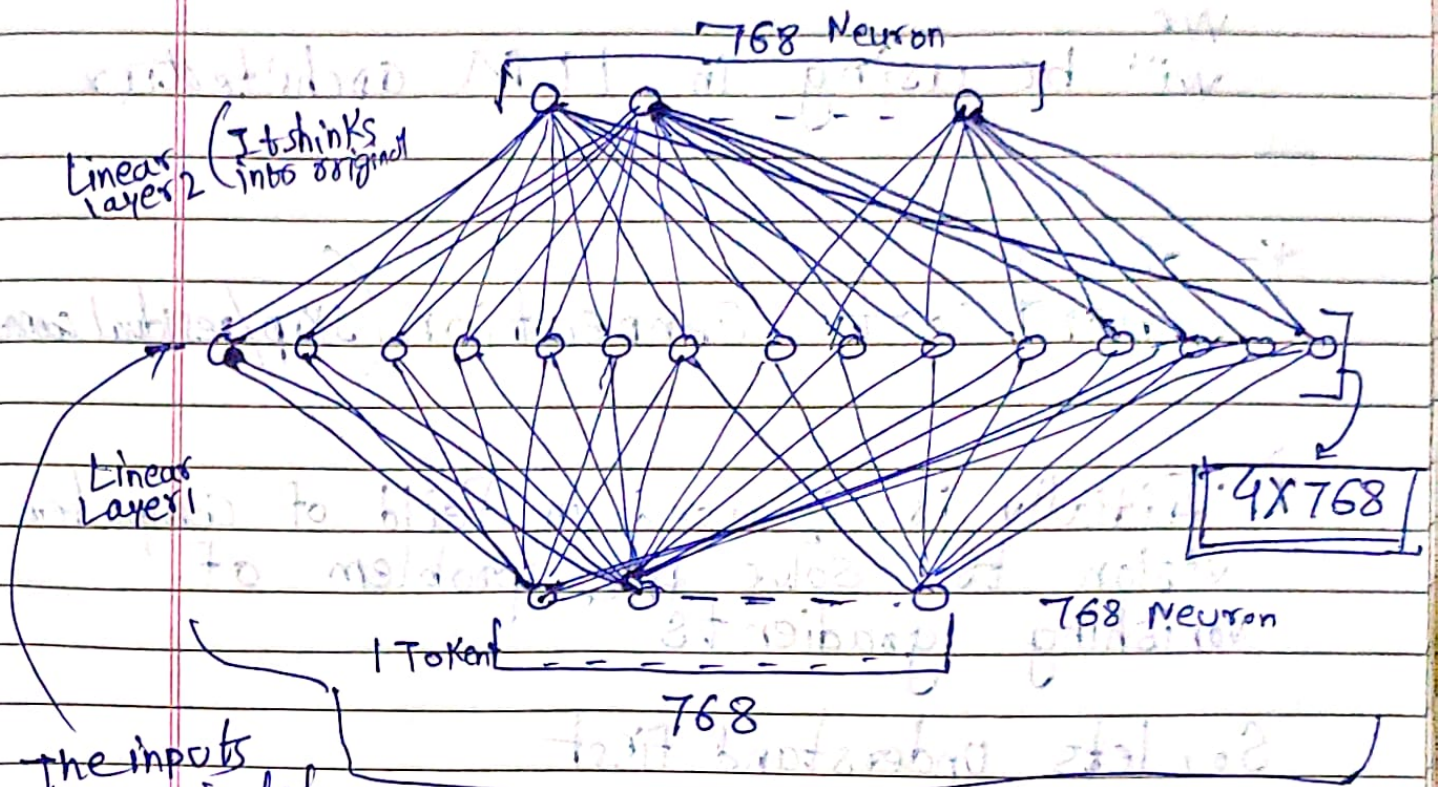
But,
we use the approximate value of GELU function is used in LLM model, which is

$$\text{GELU} \approx 0.5 * x * (1 + \tanh[\sqrt{2/\pi} * (x + 0.044715 * x^3)])$$

GELU done

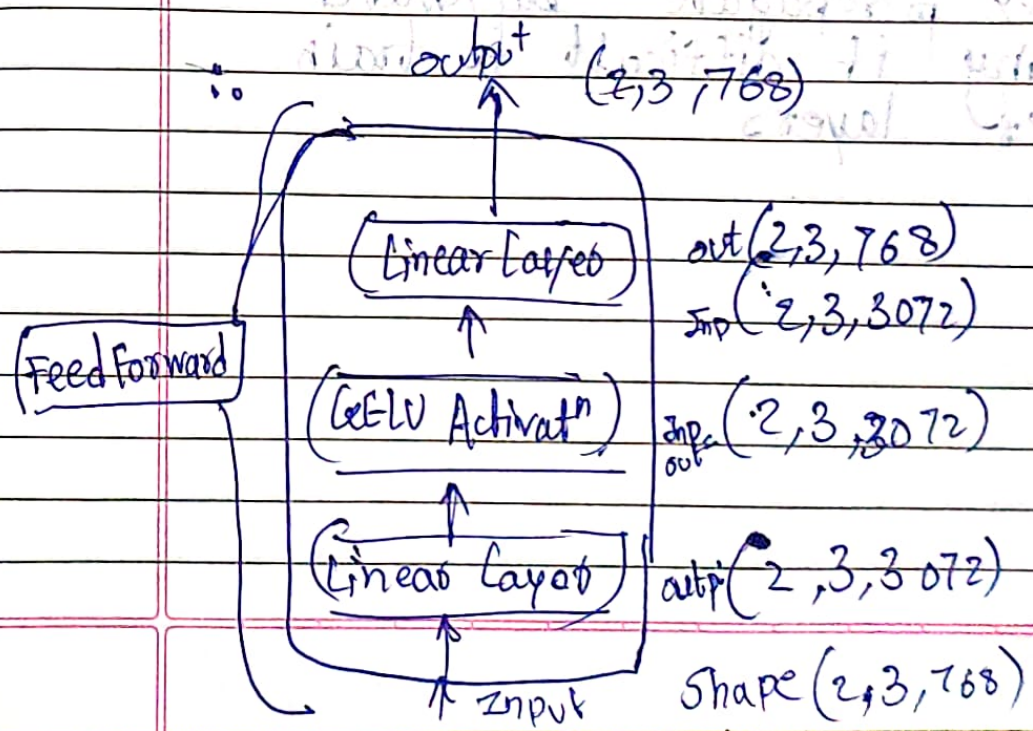


Lets see Linear layer



The inputs are projected into a 4-times larger space via the first linear layer

Expansion-contraction allows for a rich exploration space



PAGE No.
 DATE / /

Using the GELU activation Function, let us implement the small neural Network module: Feed Forward that we will be using in LLM architecture

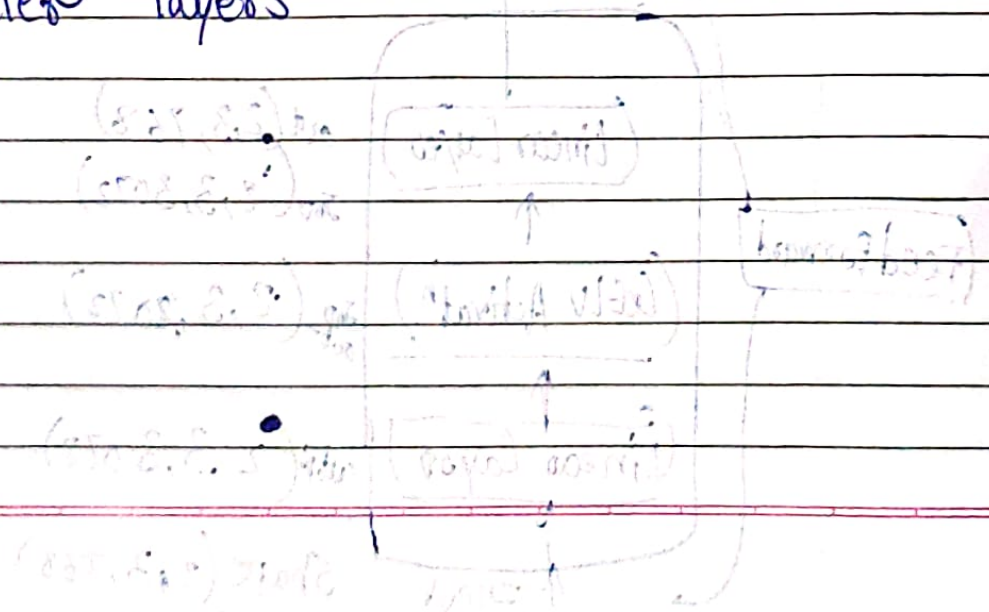
* Shortcut connection (skip/residual connect)

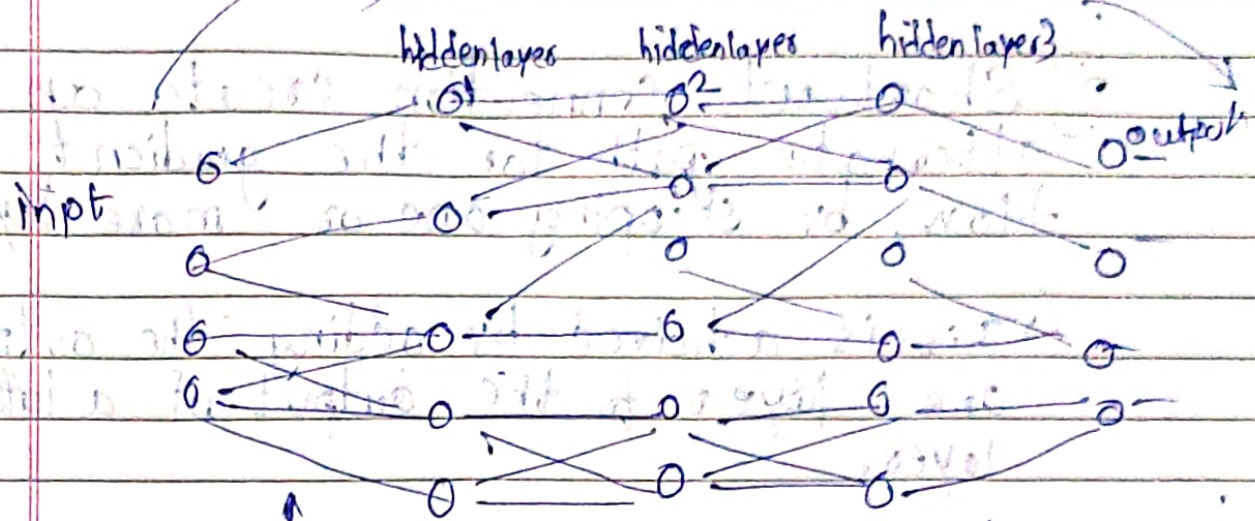
1) Initially it is used in field of computer vision to solve the problem of vanishing gradients

So, let's understand first

Vanishing Gradient

- Gradients become progressively smaller as they propagate backward
- Making it difficult to train earlier layers





gradient 1

gradient 2

gradient 3

gradient 4

It is
Matrix of
all gradient
in this layer

Gradient Flow

gradient/weight
update rule

$$W_{\text{new}} = W_{\text{old}} - \alpha \frac{dl}{dw}$$

$$\frac{dl}{dw} = \frac{d(\text{loss})}{d(\text{weight})} = \text{gradient}$$

α : step size

w: weight

$\therefore$ if Gradient become small,
eventually it will become 0, (As gradient 1, 2, 3, 4)

So,

$$\text{gradient} = 0 \rightarrow$$

$$\therefore W_{\text{new}} = W_{\text{old}}$$

$\therefore$ No new learning by nn

This is called Vanish gradient problem

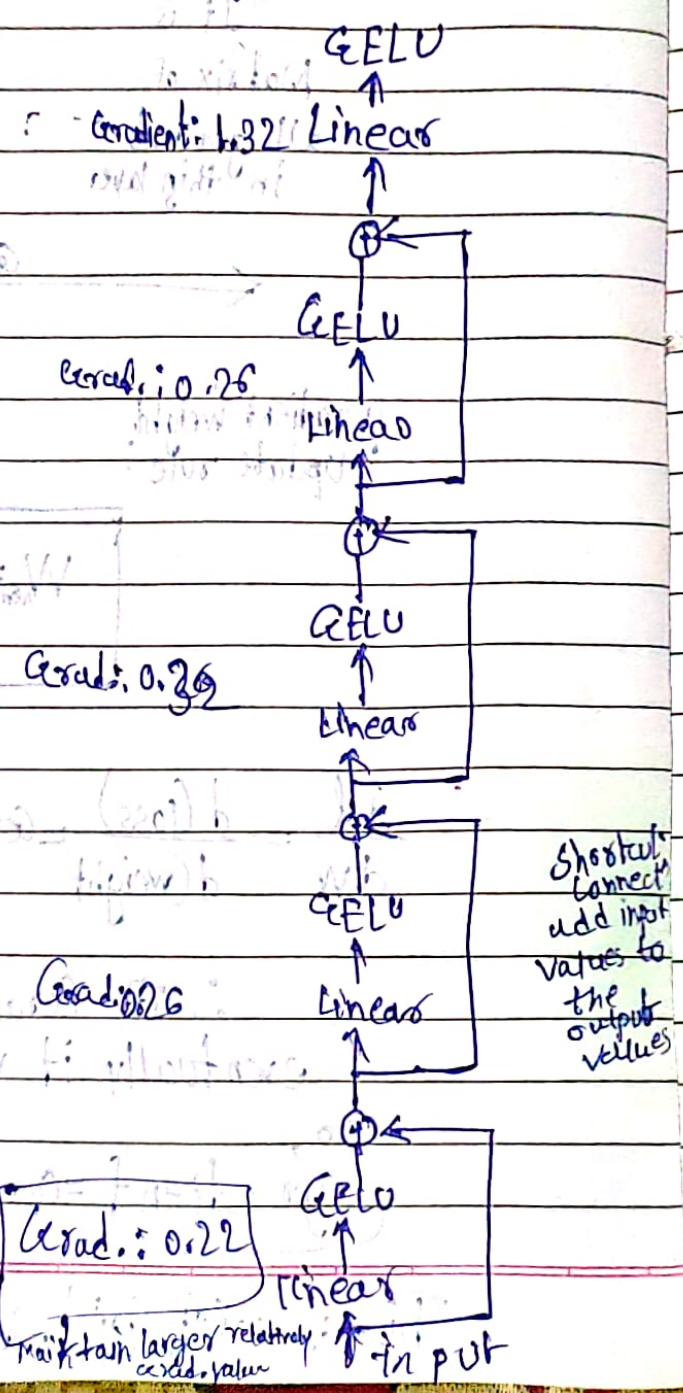
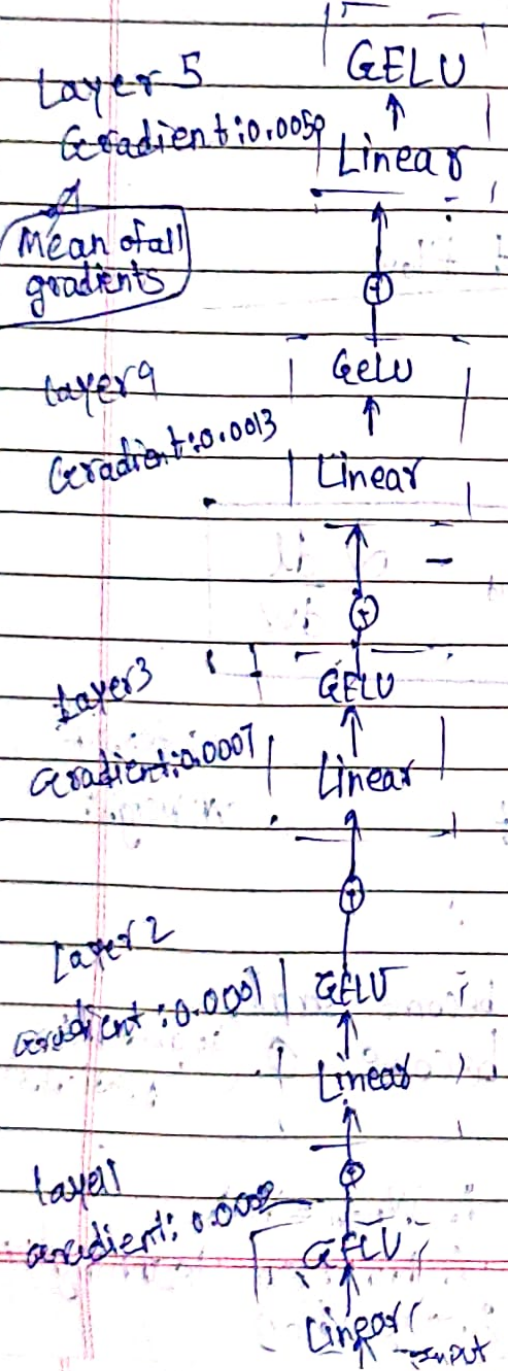
$\therefore$ Stagnation

2] Shortcut connection create an alternate path for the gradient to flow, by skipping one or more layers.

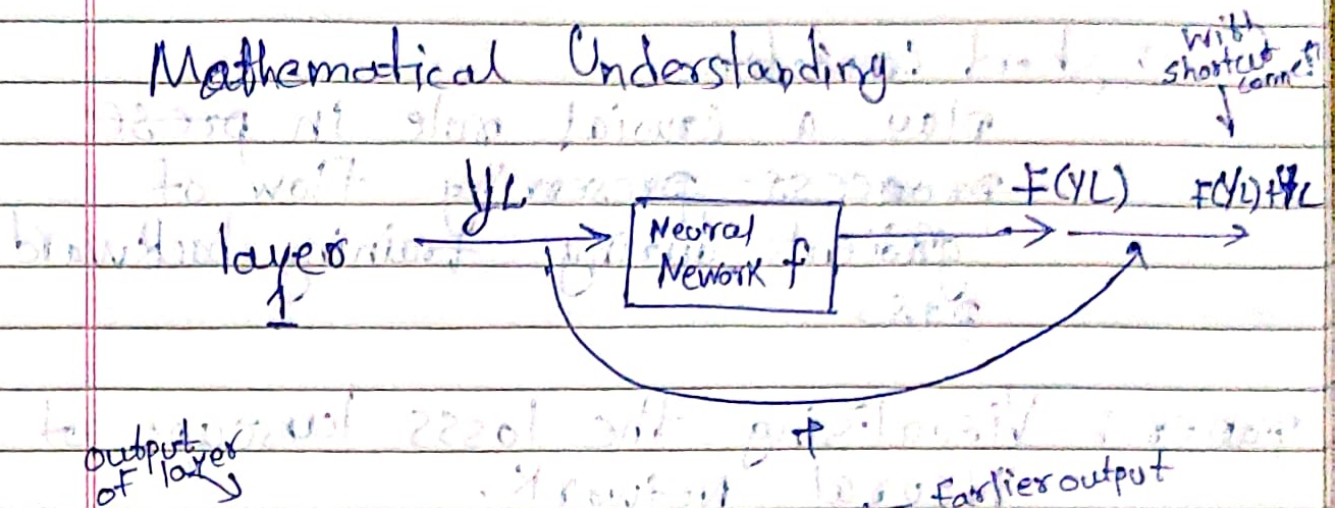
↳ This is achieved by adding the output of one layer to the output of a latter layer.

Deep Neural Network

Deep Neural Network with shortcut connect



Mathematical Understanding:



$$y_{L+1} = f(y_L) + y_L \quad \text{--- (1)}$$

$$\frac{dL}{dy_L} = \frac{dL}{dy_{L+1}} \left(\frac{dy_{L+1}}{dy_L} \right)$$

$$= \frac{dL}{dy_{L+1}} \left(\frac{df(y_L)}{dy_L} + \frac{dy_L}{dy_L} \right)$$

$$\frac{dL}{dw} = \frac{dL}{dy_{L+1}} \left(\frac{df(y_L)}{dy_L} + 1 \right)$$

Partial derivative of loss w.r.t weight of first layer

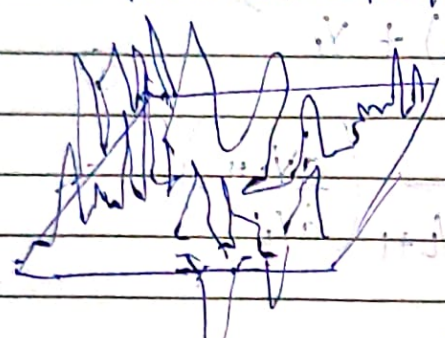
So, it is accumulated even if it reaches 0

Keep the gradient flowing through the network

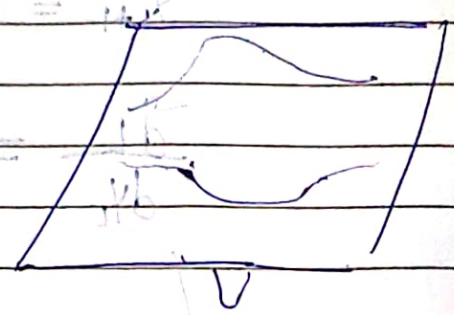
No vanish gradient problem
Stagnation

∴ Short connection
play a crucial role in prese
~~process~~ preserving flow of
gradients during training backward
pass

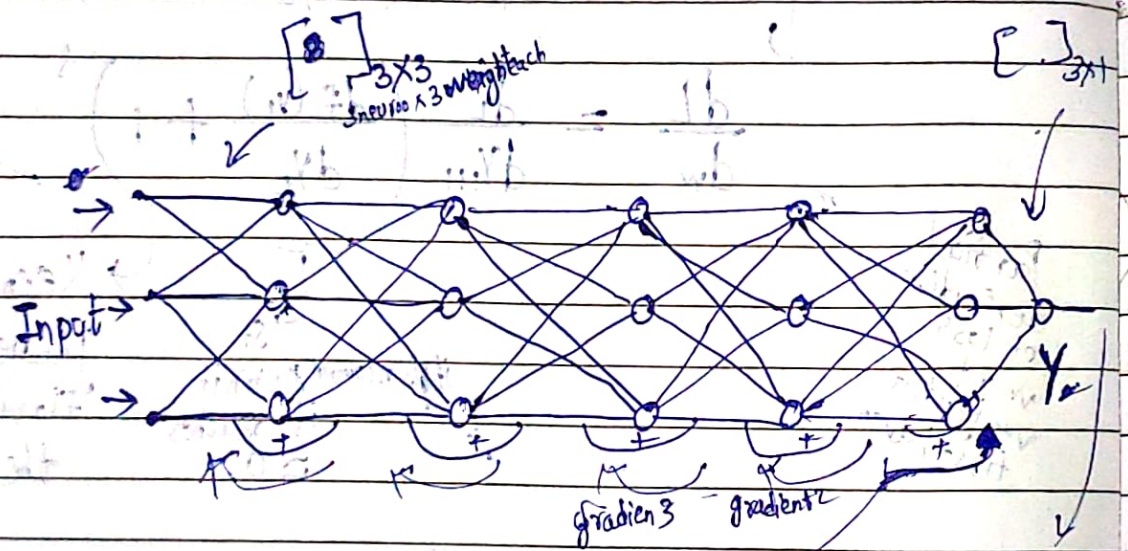
Paper: Visualising the loss landscape of
Neural Network.



Without skip connect



With skip connect



$$L = (y - \hat{y})^2$$

then take out
the mean of
gradient 1 (gradient matrix)
∴ Absolute mean
gradient

$\frac{dL}{dW}$
gradient 1
L: Loss
o: ground truth
Y: output

- | | |
|----------|-----|
| PAGE No. | |
| DATE | / / |
- 1) We have predefined output
 - 2) We have output predicted by our model
 - 3) We define loss function based on our output & True Value
 - 4) Then we find gradients of that loss, with all the parameter
 - 5) Then we update the gradients & that is how model gets better, & better & better
 - 6) And it tries to reach lower values of loss

Self attention block: Analyze relationship between input elements

Feed Forward Network: Modifies data individually at each position

The operation within Transformer block such as multi-head attention & feed forward layers, are designed to transform the input vectors such that their dimensionality is preserved.

∴ Easier to stack transformer.

So ya

768

GPT
ModelLinear output
Layers

Final LayerNorm

Transformers

Dropout

Feed Forward

Layer Norm2

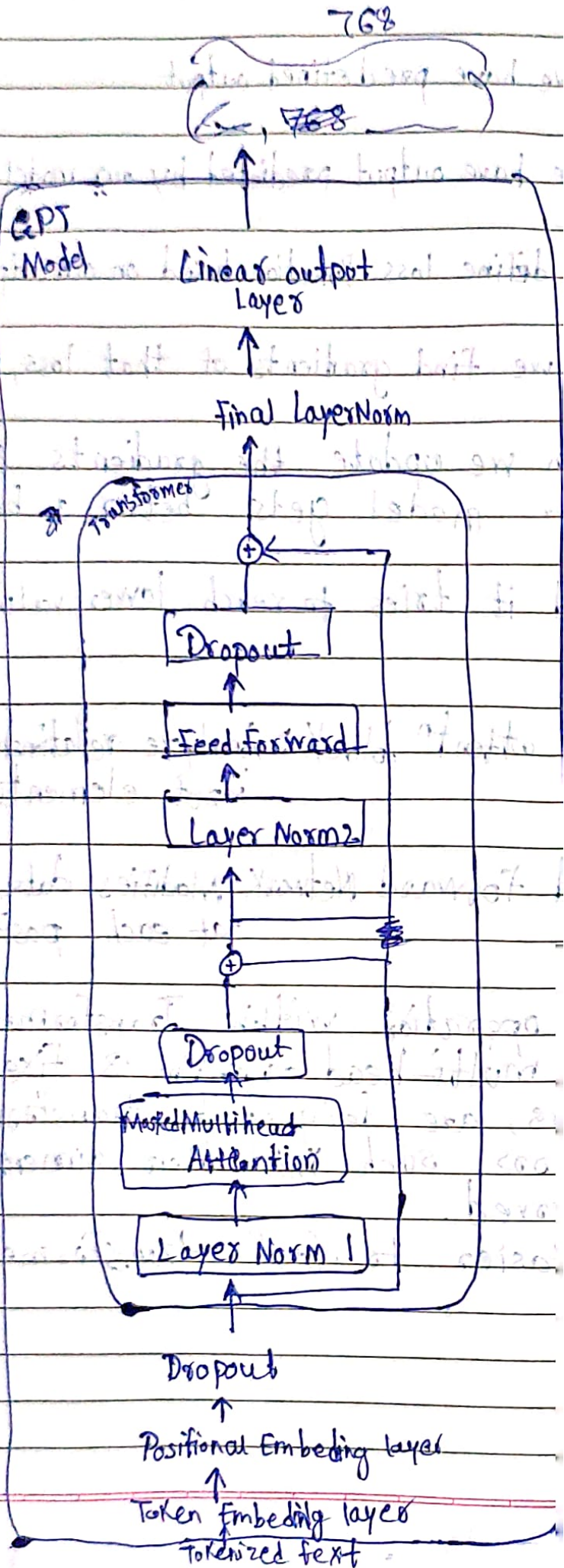
Dropout

Masked Multihead
Attention

Layer Norm 1

Dropout

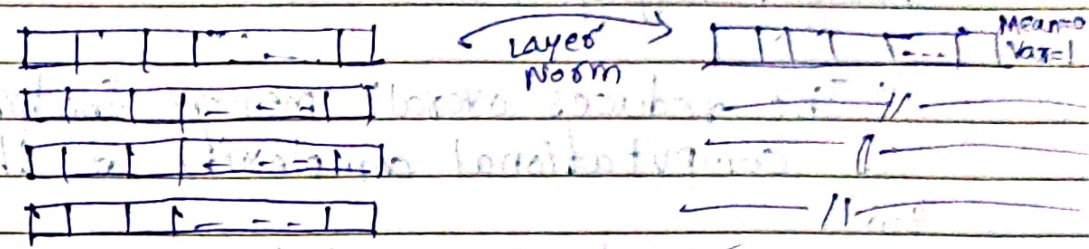
Positional Embedding layer

Token Embedding layer
Tokenized text

Post transformer

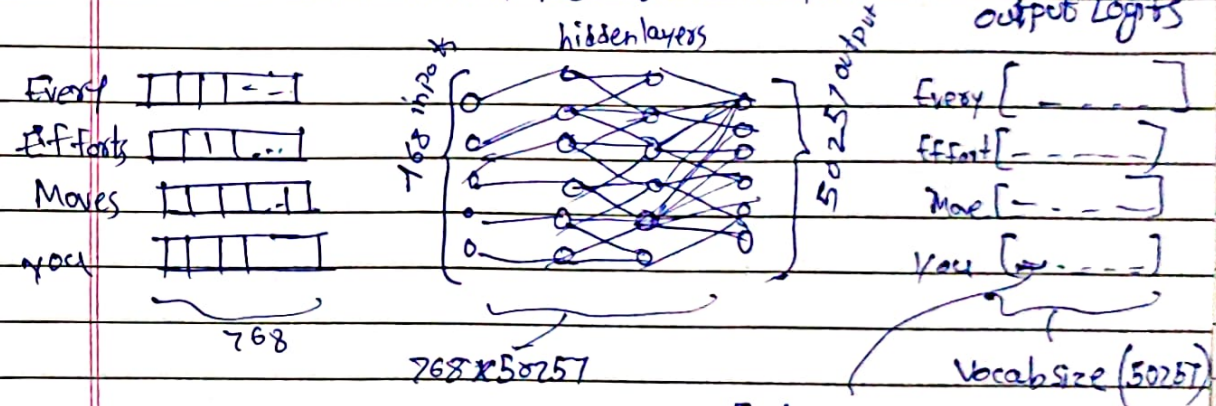
1) Layer Normalization

Input (transformer's output)



2) Linear output layer (output head)

It is neural network th last.



Each element here corresponds to the probability of it being the next token.

Entire Transformer is in Vocab

In GPT-2 Arch,

Parameters used for Token embedding layer were,

the same used in output head layer.

This is

Weight tying.

↳ This reduces overall memory footprint & computational complexity of the model. But

Using separate token emb layers & output layer results in better training & model performance.

So we are using separate layers in our GPT model implementation. &

It is true for modern LLMs.

Predicting Next word :

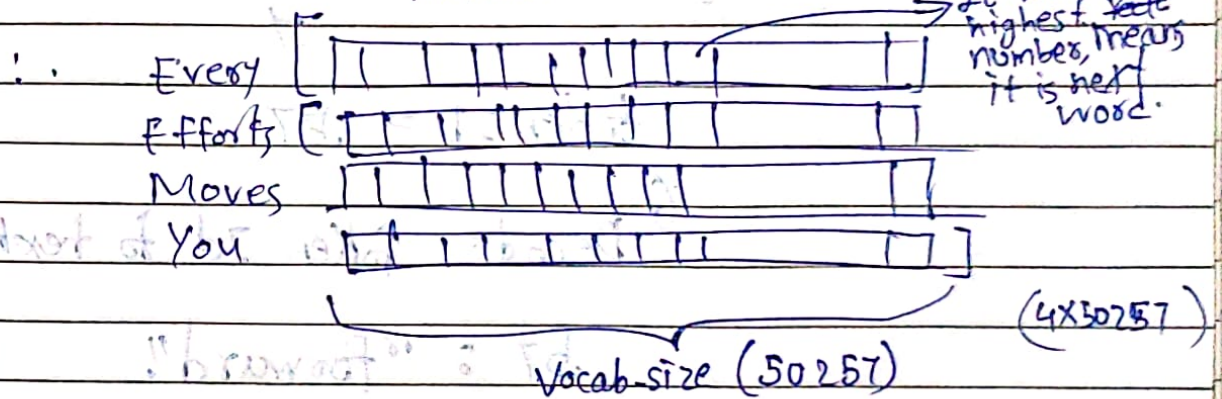
1st iteration: Hello. The next generated token
 2nd iteration: Hello,
 3rd iteration: Hello, ~~the~~ I
 4th iteration: Hello, I am

So, we have
 Output, of GPU Arch,
 which is logits Matrix
 which gives;

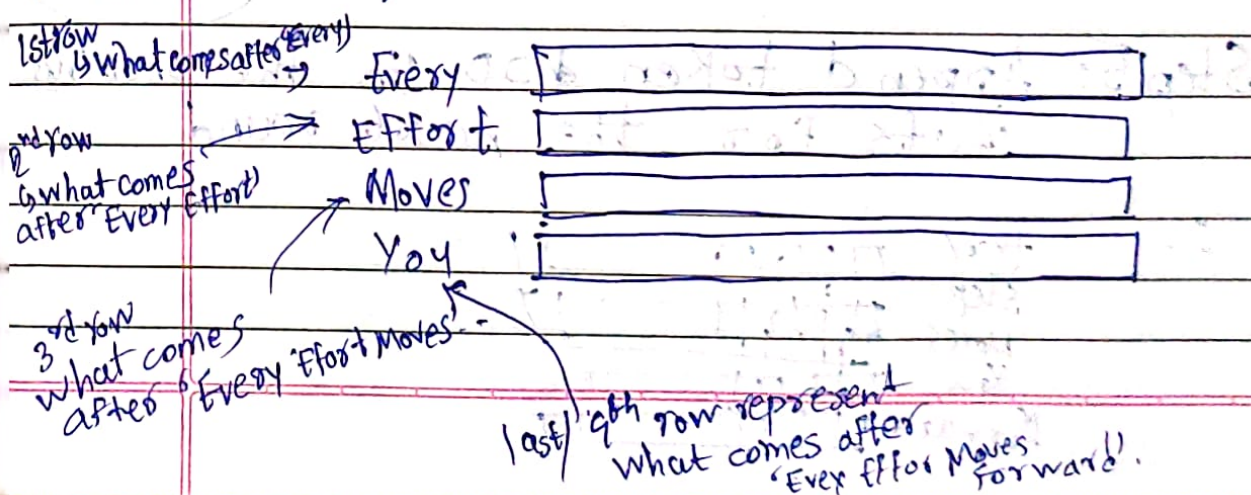
if batch=1: $1 \times 4 \times 50257$

batch=2: $2 \times 4 \times 50257$

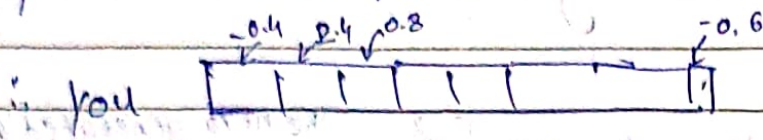
$\text{batchsize} \times \text{Context Len} \times \text{Vocab-size}$
 No. of token



Step 1

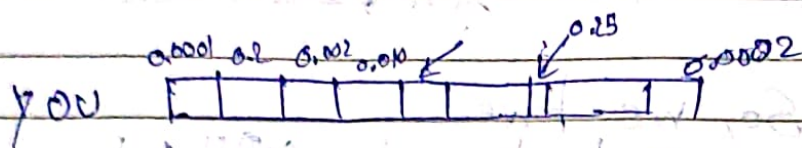


Step 2: Extract last vector



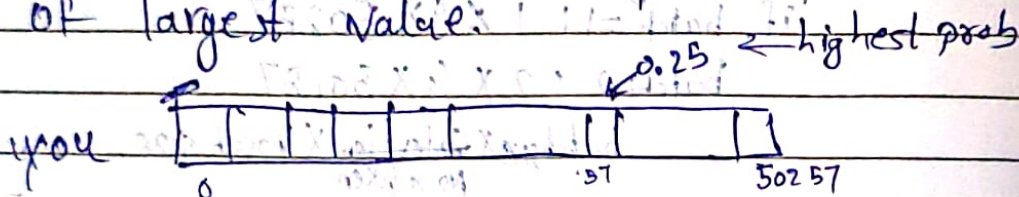
Sum Not Equal = 1
∴ Can't use Probab

Step 3: Convert logits into probabilities by applying ~~for~~ Softmax



Probab

Step 4: Identify the index position (token ID) of largest value:



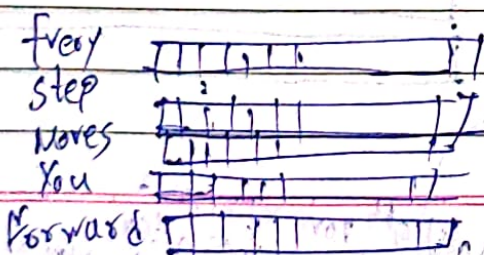
token ID = 57

Decode token ID to text

57 : "forward"

(uptil Now we Not trained LM so it will be random word)

Step 5: Append token ID to the previous inputs for the next round



In previous section

↳ GPT model output tensors with shape:

batch size $\times$ Num of tokens $\times$ vocab size

$\left(\begin{bmatrix} E \\ E \\ E \end{bmatrix} \right)$ GPT output
↓
[, , , ,] last token's logits

↓
Softmax

[0.001, 0.000, 0.21, 0.00]

↓
0 376 50257

↓
decode

Forward

Append token to the previous inputs for the next round

* In each step, the model outputs a matrix with vectors representing the potential next token

* The vector corresponding to the next token is extracted & converted into a probability distribution via the softmax

Index of highest value → Token ID → Decode back to text

Produce next token

Appended to previous inputs

This step-by-step process enables the model to generate text sequentially, building coherent phrases from the initial input context.

In practice, we repeat this process over many iterations, until we reach a user specified Number of generated tokens

(So, we have to specify, how many tokens it can use, & when to stop.

Input provided by user	Iteration	Token ID
	1	[1549, 11, 314, 716] - Predict → [257] a
	2	[1549, 11, 314, 716, 257] - Predict → [2746] model
	3	[1549, 11, 314, 716, 257, 2746] - Predict →
	6	[1549, 11, 314, 716, 257, 2746, 3442, 284, 13] - Predict → The max new tokens = 6

Code in VSCode, will be provided.

Although, we don't require softmax really, but it will be useful further, in addition sampling the techniques, where we modify softmax outputs,

such that

the model doesn't always select the most likely token, which introduces variability & creativity in the generated text.

We implemented the GPT architecture & initialized a GPT model instance with initial random weights.

So ya,

